

Mechanistic Interpretability & Simple Games

Budapest Semesters in Mathematics Undergraduate Research

Vasudev Menon Sam Reiter Jayson Tran Quyen Tran
Benjamin Whitsett Pál Zsámboki

December 11, 2025

1 Transformer Recap

Sequences, embeddings, attention mechanics

2 The (n, k) Row-Completion Task

Board states, weak/strong task, research questions

3 Theoretical Results

BOS, explicit solutions, limit behavior

4 Experimental Verification

Training pipeline, learned structure, failure modes

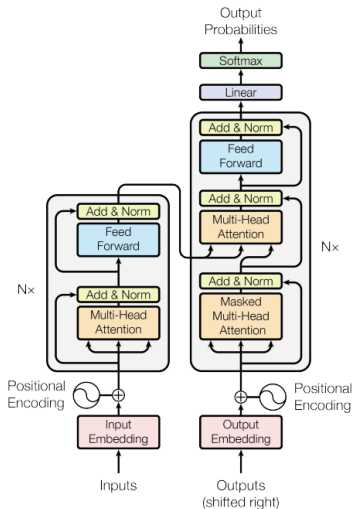
5 Further Results

Dimensionality reduction, large-scale models, conclusions

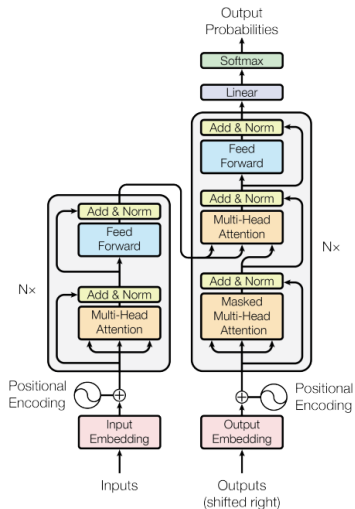
Recap of Transformers

What is a Transformer? (Overview)

- A transformer is a machine learning (ML) algorithm that acts on a **sequence** of tokens.

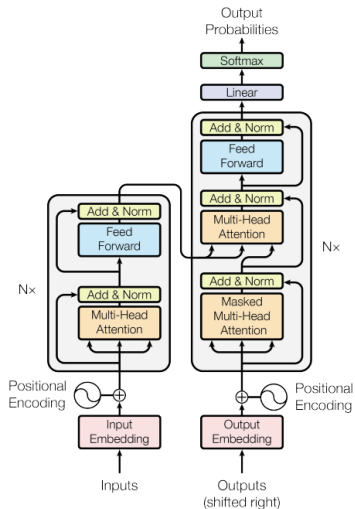


What is a Transformer? (Overview)



- A transformer is a machine learning (ML) algorithm that acts on a **sequence** of tokens.
- Each token is mapped to a vector in $\mathbb{R}^d$ (an *embedding*).

What is a Transformer? (Overview)



- A transformer is a machine learning (ML) algorithm that acts on a **sequence** of tokens.
- Each token is mapped to a vector in $\mathbb{R}^d$ (an *embedding*).
- The ML algorithm consists of repeated layers that:
 - ▶ let tokens “talk to each other” via *attention*
 - ▶ apply a simple nonlinear map to each position (feed-forward network).

Sequences as Matrices

- A sequence of n tokens is represented as a matrix

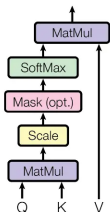
$$X = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \in \mathbb{R}^{n \times d},$$

where the i -th row $x_i \in \mathbb{R}^d$ is the embedding of token i .

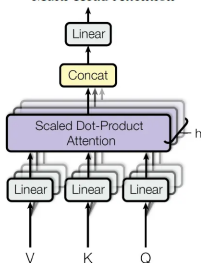
- Transformer layers operate on X using:
 - matrix multiplications,
 - elementwise nonlinearities,
 - and row-wise normalizations.

Attention as a Matrix Operator

Scaled Dot-Product Attention



Multi-Head Attention

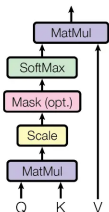


- Given a sequence matrix $X \in \mathbb{R}^{n \times d}$, form:

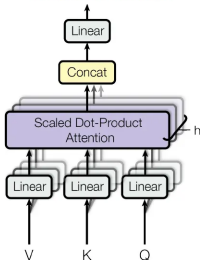
$$Q = XW_Q, K = XW_K, V = XW_V.$$

Attention as a Matrix Operator

Scaled Dot-Product Attention



Multi-Head Attention



- Given a sequence matrix $X \in \mathbb{R}^{n \times d}$, form:

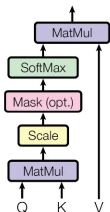
$$Q = XW_Q, K = XW_K, V = XW_V.$$

- Computes a score matrix

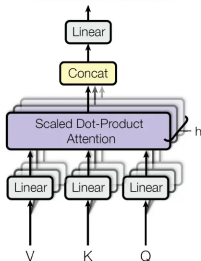
$$A(X) = XW_QW_K^T X^T \in \mathbb{R}^{n \times n}.$$

Attention as a Matrix Operator

Scaled Dot-Product Attention



Multi-Head Attention



- Given a sequence matrix $X \in \mathbb{R}^{n \times d}$, form:

$$Q = XW_Q, K = XW_K, V = XW_V.$$

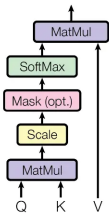
- Computes a score matrix

$$A(X) = XW_QW_K^T X^T \in \mathbb{R}^{n \times n}.$$

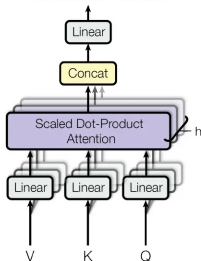
- Applying normalization: $\sigma(A)$

Updating the Sequence via Attention

Scaled Dot-Product Attention



Multi-Head Attention

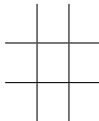


- Define $U = W_V$ and use the score matrix $A = \sigma(XW_QW_K^T X^T)$.
- The attention update is

$$X' = XU + AXW_QW_KU.$$

Our Research Questions

Tic Tac Toe



Tic Tac Toe

Label the board with $i \in [9]$

1	2	3
4	5	6
7	8	9

Game States

Enumerate played moves and detect when a three in a row is created.

1	2	3

- played moves: {1,2,3}
- three in a row: True

Game States

Enumerate played moves and detect when a three in a row is created.

1	2	3

- played moves: $\{1,2,3\}$
- three in a row: True
- Note that we do not consider column/diagonal completions as three in a *row*

Game States

1	2	
	5	
7		

- played moves: {1,2,5,7}
- three in a row: False

Game States

1	2	
7	8	9

- played moves: {1,2,7,8,9}
- three in a row: True

Generalization

In fact, we generalize to any board with n positions with k columns (where $k \mid n$).

Generalization

In fact, we generalize to any board with n positions with k columns (where $k \mid n$). Our goal then is to detect when a “ k in a row” occurs.

Generalization

In fact, we generalize to any board with n positions with k columns (where $k \mid n$). Our goal then is to detect when a “ k in a row” occurs. For example, in tic-tac-toe, $n = 9$ and $k = 3$.

Generalization

In fact, we generalize to any board with n positions with k columns (where $k \mid n$). Our goal then is to detect when a “ k in a row” occurs. For example, in tic-tac-toe, $n = 9$ and $k = 3$. Formally, we divide this into two tasks:

Generalization

In fact, we generalize to any board with n positions with k columns (where $k \mid n$). Our goal then is to detect when a “ k in a row” occurs. For example, in tic-tac-toe, $n = 9$ and $k = 3$. Formally, we divide this into two tasks:

- Weak Task: Detect when the *first* win occurs.

Generalization

In fact, we generalize to any board with n positions with k columns (where $k \mid n$). Our goal then is to detect when a “ k in a row” occurs. For example, in tic-tac-toe, $n = 9$ and $k = 3$. Formally, we divide this into two tasks:

- Weak Task: Detect when the *first* win occurs.
- Strong Task: Detect where *every* win just occurred.

Strong Task: Detect All Wins

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$ with $s_t \in [n]$, detect *every time step* t where a winning set of k contiguous positions is completed.

Strong Task: Detect All Wins

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$ with $s_t \in [n]$, detect *every time step* t where a winning set of k contiguous positions is completed.

Strong Task: Detect All Wins

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$ with $s_t \in [n]$, detect *every time step* t where a winning set of k contiguous positions is completed.

Formally, define an indicator vector $v \in \{0, 1\}^n$ as

$$v_t = \begin{cases} 1 & \text{if } B(s_t) \subseteq \{s_1, \dots, s_t\} \\ 0 & \text{otherwise} \end{cases}$$

Where $B(x)$ is the partition block containing x : $B(x) = \{y \mid \lceil y/k \rceil = \lceil x/k \rceil\}$.

Strong Task: Detect All Wins

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$ with $s_t \in [n]$, detect *every time step* t where a winning set of k contiguous positions is completed.

Formally, define an indicator vector $v \in \{0, 1\}^n$ as

$$v_t = \begin{cases} 1 & \text{if } B(s_t) \subseteq \{s_1, \dots, s_t\} \\ 0 & \text{otherwise} \end{cases}$$

Where $B(x)$ is the partition block containing x : $B(x) = \{y \mid \lceil y/k \rceil = \lceil x/k \rceil\}$.

Examples (assuming $n = 9$ and $k = 3$):

Strong Task: Detect All Wins

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$ with $s_t \in [n]$, detect *every time step* t where a winning set of k contiguous positions is completed.

Formally, define an indicator vector $v \in \{0, 1\}^n$ as

$$v_t = \begin{cases} 1 & \text{if } B(s_t) \subseteq \{s_1, \dots, s_t\} \\ 0 & \text{otherwise} \end{cases}$$

Where $B(x)$ is the partition block containing x : $B(x) = \{y \mid \lceil y/k \rceil = \lceil x/k \rceil\}$.

Examples (assuming $n = 9$ and $k = 3$):

- $S = (1, 2, 3, 4, 5, 6, 7, 8, 9) \rightarrow v = [0, 0, 1, 0, 0, 1, 0, 0, 1]$

Strong Task: Detect All Wins

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$ with $s_t \in [n]$, detect *every time step* t where a winning set of k contiguous positions is completed.

Formally, define an indicator vector $v \in \{0, 1\}^n$ as

$$v_t = \begin{cases} 1 & \text{if } B(s_t) \subseteq \{s_1, \dots, s_t\} \\ 0 & \text{otherwise} \end{cases}$$

Where $B(x)$ is the partition block containing x : $B(x) = \{y \mid \lceil y/k \rceil = \lceil x/k \rceil\}$.

Examples (assuming $n = 9$ and $k = 3$):

- $S = (1, 2, 3, 4, 5, 6, 7, 8, 9) \rightarrow v = [0, 0, 1, 0, 0, 1, 0, 0, 1]$
- $S = (1, 2, 7, 8, 9, 4, 3, 6, 5) \rightarrow v = [0, 0, 0, 0, 1, 0, 1, 0, 1]$

Strong Task: Detect All Wins

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$ with $s_t \in [n]$, detect *every time step* t where a winning set of k contiguous positions is completed.

Formally, define an indicator vector $v \in \{0, 1\}^n$ as

$$v_t = \begin{cases} 1 & \text{if } B(s_t) \subseteq \{s_1, \dots, s_t\} \\ 0 & \text{otherwise} \end{cases}$$

Where $B(x)$ is the partition block containing x : $B(x) = \{y \mid \lceil y/k \rceil = \lceil x/k \rceil\}$.

Examples (assuming $n = 9$ and $k = 3$):

- $S = (1, 2, 3, 4, 5, 6, 7, 8, 9) \rightarrow v = [0, 0, 1, 0, 0, 1, 0, 0, 1]$
- $S = (1, 2, 7, 8, 9, 4, 3, 6, 5) \rightarrow v = [0, 0, 0, 0, 1, 0, 1, 0, 1]$

Goal: Report all winning sets of k contiguous positions.

Weak Task: Detect the First Win

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$, detect the *first time step* t where a winning set of k contiguous positions is completed.

Weak Task: Detect the First Win

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$, detect the *first time step* t where a winning set of k contiguous positions is completed.

Formally, define an indicator vector $w \in \{0, 1, \emptyset\}^n$ where:

$$w_t = \begin{cases} \emptyset & \text{if } \exists j < t \text{ such that } w_j = 1 & \text{(Game already ended)} \\ 1 & \text{if } B(s_t) \subseteq \{s_1, \dots, s_t\} & \text{(First win!)} \\ 0 & \text{otherwise} & \text{(Keep playing)} \end{cases}$$

Weak Task: Detect the First Win

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$, detect the *first time step* t where a winning set of k contiguous positions is completed.

Formally, define an indicator vector $w \in \{0, 1, \emptyset\}^n$ where:

$$w_t = \begin{cases} \emptyset & \text{if } \exists j < t \text{ such that } w_j = 1 & \text{(Game already ended)} \\ 1 & \text{if } B(s_t) \subseteq \{s_1, \dots, s_t\} & \text{(First win!)} \\ 0 & \text{otherwise} & \text{(Keep playing)} \end{cases}$$

Examples (assuming $n = 9$ and $k = 3$):

Weak Task: Detect the First Win

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$, detect the *first time step* t where a winning set of k contiguous positions is completed.

Formally, define an indicator vector $w \in \{0, 1, \emptyset\}^n$ where:

$$w_t = \begin{cases} \emptyset & \text{if } \exists j < t \text{ such that } w_j = 1 & \text{(Game already ended)} \\ 1 & \text{if } B(s_t) \subseteq \{s_1, \dots, s_t\} & \text{(First win!)} \\ 0 & \text{otherwise} & \text{(Keep playing)} \end{cases}$$

Examples (assuming $n = 9$ and $k = 3$):

- $S = (1, 2, 3, 4, 5, 6, 7, 8, 9) \rightarrow w = [0, 0, 1, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset]$

Weak Task: Detect the First Win

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$, detect the *first time step* t where a winning set of k contiguous positions is completed.

Formally, define an indicator vector $w \in \{0, 1, \emptyset\}^n$ where:

$$w_t = \begin{cases} \emptyset & \text{if } \exists j < t \text{ such that } w_j = 1 & \text{(Game already ended)} \\ 1 & \text{if } B(s_t) \subseteq \{s_1, \dots, s_t\} & \text{(First win!)} \\ 0 & \text{otherwise} & \text{(Keep playing)} \end{cases}$$

Examples (assuming $n = 9$ and $k = 3$):

- $S = (1, 2, 3, 4, 5, 6, 7, 8, 9) \rightarrow w = [0, 0, 1, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset]$
- $S = (1, 2, 7, 8, 9, 4, 3, 6, 5) \rightarrow w = [0, 0, 0, 0, 1, \emptyset, \emptyset, \emptyset, \emptyset]$

Weak Task: Detect the First Win

Problem: Given a sequence of played moves $S = (s_1, \dots, s_n)$, detect the *first time step* t where a winning set of k contiguous positions is completed.

Formally, define an indicator vector $w \in \{0, 1, \emptyset\}^n$ where:

$$w_t = \begin{cases} \emptyset & \text{if } \exists j < t \text{ such that } w_j = 1 & \text{(Game already ended)} \\ 1 & \text{if } B(s_t) \subseteq \{s_1, \dots, s_t\} & \text{(First win!)} \\ 0 & \text{otherwise} & \text{(Keep playing)} \end{cases}$$

Examples (assuming $n = 9$ and $k = 3$):

- $S = (1, 2, 3, 4, 5, 6, 7, 8, 9) \rightarrow w = [0, 0, 1, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset]$
- $S = (1, 2, 7, 8, 9, 4, 3, 6, 5) \rightarrow w = [0, 0, 0, 0, 1, \emptyset, \emptyset, \emptyset, \emptyset]$

Goal: Detect the first win.

Our Research Questions

Our Research Questions

We aim to train a Transformer model that takes in the board state and outputs the “indicator” vector to detect if a k in a row has happened.

Our Research Questions

We aim to train a Transformer model that takes in the board state and outputs the “indicator” vector to detect if a k in a row has happened.

Questions:

Our Research Questions

We aim to train a Transformer model that takes in the board state and outputs the “indicator” vector to detect if a k in a row has happened.

Questions:

- What criteria do all models satisfy that solve these tasks share?

Our Research Questions

We aim to train a Transformer model that takes in the board state and outputs the “indicator” vector to detect if a k in a row has happened.

Questions:

- What criteria do all models satisfy that solve these tasks share?
- What is the theoretical minimal such model?

Our Research Questions

We aim to train a Transformer model that takes in the board state and outputs the “indicator” vector to detect if a k in a row has happened.

Questions:

- What criteria do all models satisfy that solve these tasks share?
- What is the theoretical minimal such model?
- What techniques can we employ to train the minimal model?

Theoretical Discoveries

Beginning-Of-Sequence Tokens

Beginning-Of-Sequence Tokens

- Attention selects a convex combination of the input tokens

Beginning-Of-Sequence Tokens

- Attention selects a convex combination of the input tokens
- Each single token is a loss (< 0)

Beginning-Of-Sequence Tokens

- Attention selects a convex combination of the input tokens
- Each single token is a loss (< 0)
- $\implies$ *every* sequence will look like a loss!

Beginning-Of-Sequence Tokens

- Attention selects a convex combination of the input tokens
- Each single token is a loss (< 0)
- $\implies$ *every* sequence will look like a loss!
- $\implies$ A single-layer transformer doesn't work, unless...

Beginning-Of-Sequence Tokens

- Attention selects a convex combination of the input tokens
- Each single token is a loss (< 0)
- $\implies$ *every* sequence will look like a loss!

- $\implies$ A single-layer transformer doesn't work, unless...
- we pay attention to an extra initialization token!

First Solution

First Solution

- Embedding: move in row $i \mapsto (0, \dots, \underset{i}{1}, \dots, 0, 0),$
BOS $\mapsto (0, \dots, 0, k-1)$

First Solution

- Embedding: move in row $i \mapsto (0, \dots, \underset{i}{1}, \dots, 0, 0),$
BOS $\mapsto (0, \dots, 0, k-1)$
“each row gets a basis vector”

First Solution

- Embedding: move in row $i \mapsto (0, \dots, \underset{i}{1}, \dots, 0, 0),$
BOS $\mapsto (0, \dots, 0, k-1)$
“each row gets a basis vector”
- Attention bilinear form: $(W_Q W_K^T)_{ij} = \begin{cases} 0 & \text{if } i = j \text{ or } i = [BOS], \\ -\infty & \text{otherwise.} \end{cases}$

First Solution

- Embedding: move in row $i \mapsto (0, \dots, \underset{i}{1}, \dots, 0, 0)$,
BOS $\mapsto (0, \dots, 0, k-1)$
“each row gets a basis vector”
- Attention bilinear form: $(W_Q W_K^T)_{ij} = \begin{cases} 0 & \text{if } i = j \text{ or } i = [\text{BOS}], \\ -\infty & \text{otherwise.} \end{cases}$
“each row only gets attention from itself and the BOS”

First Solution

- Embedding: move in row $i \mapsto (0, \dots, \underset{i}{1}, \dots, 0, 0)$,
BOS $\mapsto (0, \dots, 0, k-1)$
“each row gets a basis vector”
- Attention bilinear form: $(W_Q W_K^T)_{ij} = \begin{cases} 0 & \text{if } i = j \text{ or } i = [\text{BOS}], \\ -\infty & \text{otherwise.} \end{cases}$
“each row only gets attention from itself and the BOS”
- Getting output: $(W_V W_O U)_i = \begin{cases} -1 & \text{if } i = [\text{BOS}], \\ +1 & \text{otherwise.} \end{cases}$

First Solution

- Embedding: move in row $i \mapsto (0, \dots, \underset{i}{1}, \dots, 0, 0)$,
BOS $\mapsto (0, \dots, 0, k-1)$
“each row gets a basis vector”
- Attention bilinear form: $(W_Q W_K^T)_{ij} = \begin{cases} 0 & \text{if } i = j \text{ or } i = [\text{BOS}], \\ -\infty & \text{otherwise.} \end{cases}$
“each row only gets attention from itself and the BOS”
- Getting output: $(W_V W_O U)_i = \begin{cases} -1 & \text{if } i = [\text{BOS}], \\ +1 & \text{otherwise.} \end{cases}$
“moves count up, BOS weighs the sum down”

Size of the First Solution

Size of the First Solution

- Embedding dimension: $d = \frac{n}{k} + 1$ (number of rows, plus BOS dimension)

Size of the First Solution

- Embedding dimension: $d = \frac{n}{k} + 1$ (number of rows, plus BOS dimension)
- Key dimension (i.e. $\text{rk}(W_Q W_K^T)$): $d_k = \frac{n}{k}$

Size of the First Solution

- Embedding dimension: $d = \frac{n}{k} + 1$ (number of rows, plus BOS dimension)
- Key dimension (i.e. $\text{rk}(W_Q W_K^T)$): $d_k = \frac{n}{k}$
- Output dimension (i.e. $\text{rk}(W_V W_O U)$): $d_o = 1$ (one-dimensional readout)

Size of the First Solution

- Embedding dimension: $d = \frac{n}{k} + 1$ (number of rows, plus BOS dimension)
- Key dimension (i.e. $\text{rk}(W_Q W_K^T)$): $d_k = \frac{n}{k}$
- Output dimension (i.e. $\text{rk}(W_V W_O U)$): $d_o = 1$ (one-dimensional readout)
- Can we do better?

Improved Solution

Improved Solution

- WLOG index the rows by unit-incrementing values centered at 0

Improved Solution

- WLOG index the rows by unit-incrementing values centered at 0
- Embedding: move in row $i \mapsto (\cosh(i), \sinh(i))$,
BOS $\mapsto (0, 0)$

Improved Solution

- WLOG index the rows by unit-incrementing values centered at 0
- Embedding: move in row $i \mapsto (\cosh(i), \sinh(i))$,
BOS $\mapsto (0, 0)$
“embed onto a hyperbola”

Improved Solution

- WLOG index the rows by unit-incrementing values centered at 0
- Embedding: move in row $i \mapsto (\cosh(i), \sinh(i))$,
BOS $\mapsto (0, 0)$
“embed onto a hyperbola”
- Attention bilinear form: $W_Q W_K^T \propto \text{diag}(-1, 1)$

Improved Solution

- WLOG index the rows by unit-incrementing values centered at 0
- Embedding: move in row $i \mapsto (\cosh(i), \sinh(i))$,
BOS $\mapsto (0, 0)$
“embed onto a hyperbola”
- Attention bilinear form: $W_Q W_K^T \propto \text{diag}(-1, 1)$
“attention is calculated by scaled dot products with a flipped x -axis”

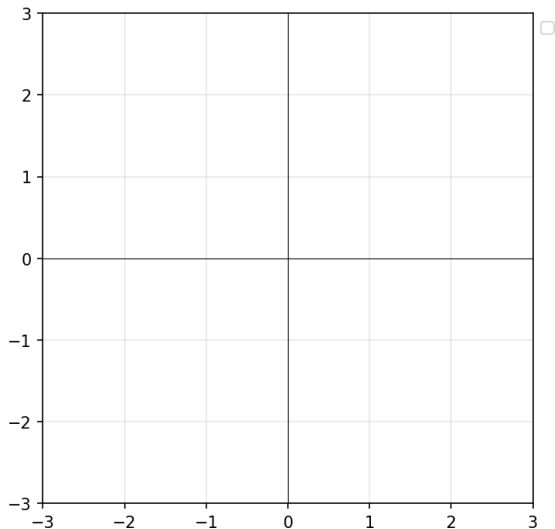
Improved Solution

- WLOG index the rows by unit-incrementing values centered at 0
- Embedding: move in row $i \mapsto (\cosh(i), \sinh(i))$,
BOS $\mapsto (0, 0)$
“embed onto a hyperbola”
- Attention bilinear form: $W_Q W_K^T \propto \text{diag}(-1, 1)$
“attention is calculated by scaled dot products with a flipped x -axis”
- Getting output: $U = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$, $W_V W_O U \propto \begin{pmatrix} 1 \\ 0 \end{pmatrix}$

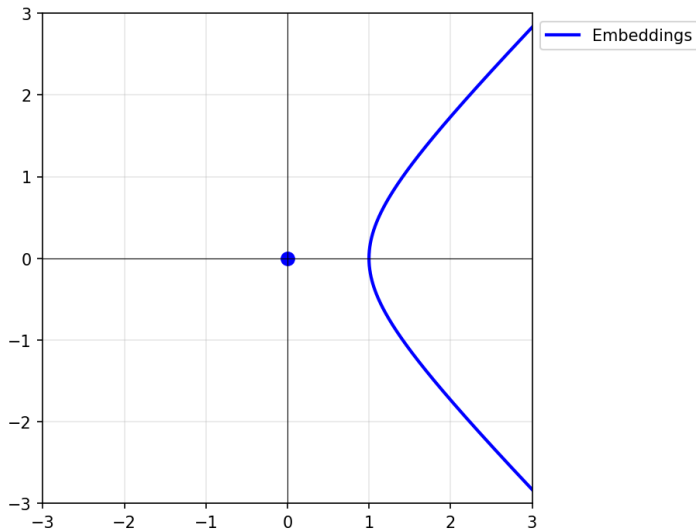
Improved Solution

- WLOG index the rows by unit-incrementing values centered at 0
- Embedding: move in row $i \mapsto (\cosh(i), \sinh(i))$,
BOS $\mapsto (0, 0)$
“embed onto a hyperbola”
- Attention bilinear form: $W_Q W_K^T \propto \text{diag}(-1, 1)$
“attention is calculated by scaled dot products with a flipped x -axis”
- Getting output: $U = \begin{pmatrix} -1 \\ 0 \end{pmatrix}$, $W_V W_O U \propto \begin{pmatrix} 1 \\ 0 \end{pmatrix}$
“lone move is a loss, but many moves raise the sum”

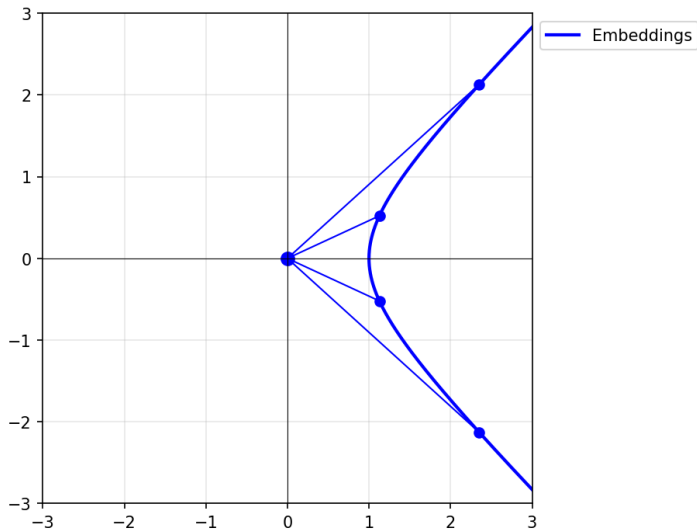
Visualizing the Improved Solution



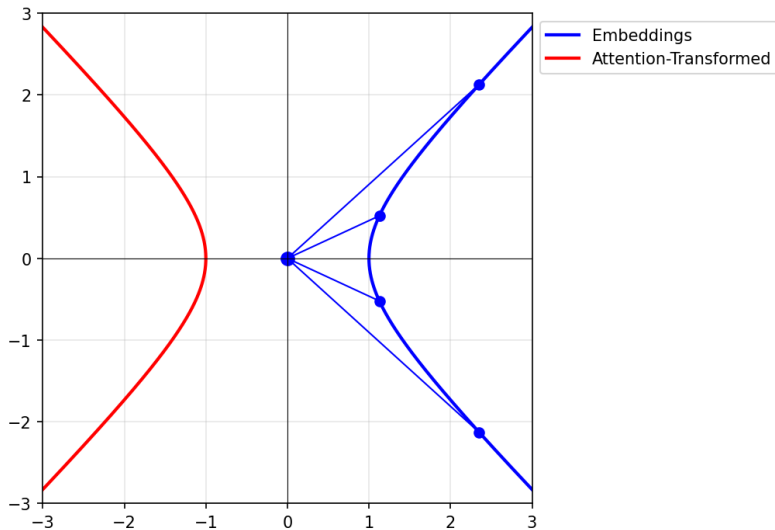
Visualizing the Improved Solution



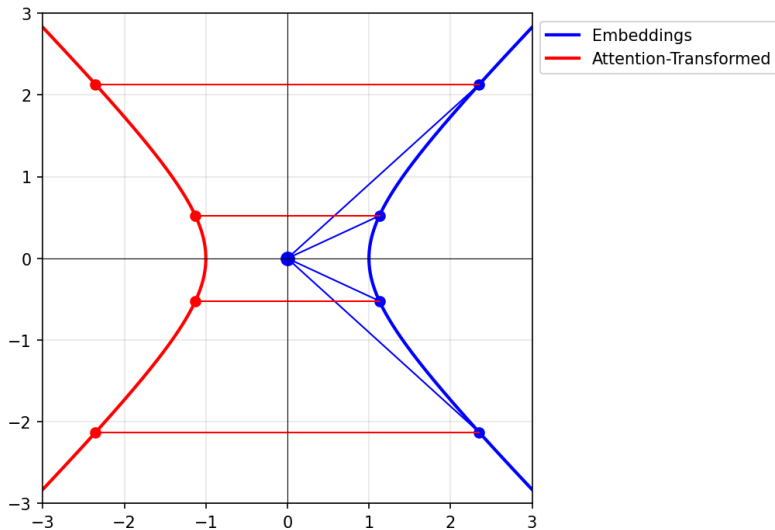
Visualizing the Improved Solution



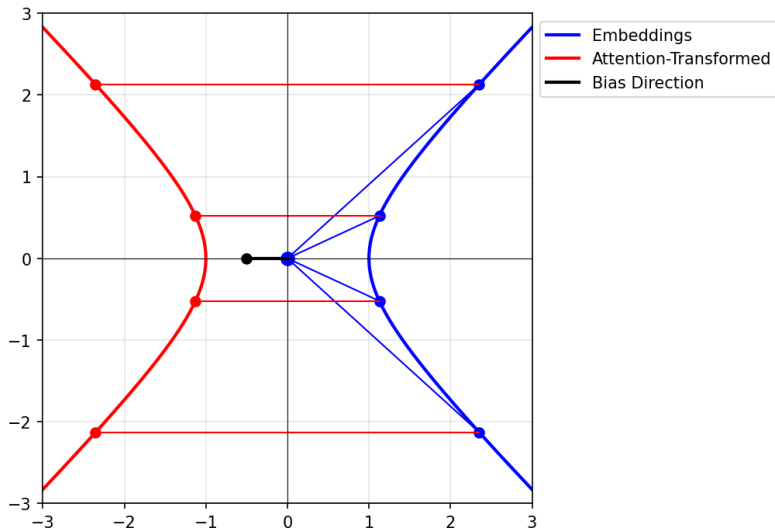
Visualizing the Improved Solution



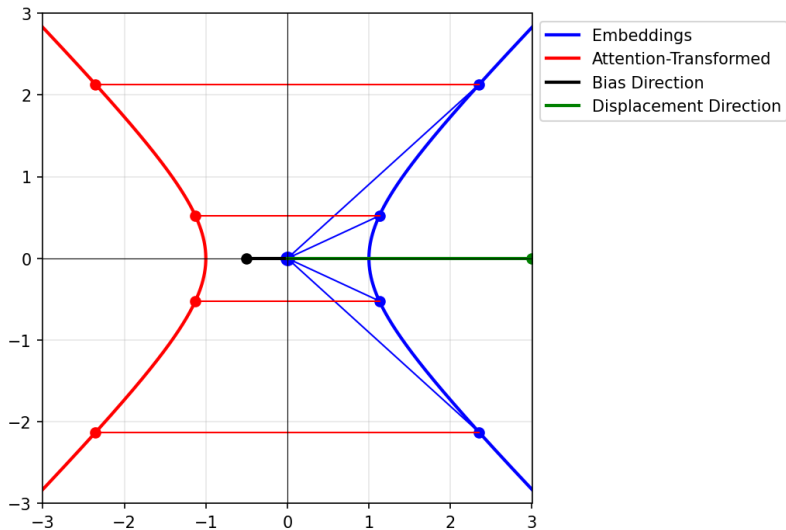
Visualizing the Improved Solution



Visualizing the Improved Solution



Visualizing the Improved Solution



“Limit” solutions ($d = 2$)

“Limit” solutions ($d = 2$)

- Fix an “embedding curve;”

“Limit” solutions ($d = 2$)

- Fix an “embedding curve;”
- Fix a BOS location;

“Limit” solutions ($d = 2$)

- Fix an “embedding curve;”
- Fix a BOS location;
- Arbitrarily assign curve points to rows;

“Limit” solutions ($d = 2$)

- Fix an “embedding curve;”
- Fix a BOS location;
- Arbitrarily assign curve points to rows;
- Is the attention pattern possibly a solution?

“Limit” solutions ($d = 2$)

- Fix an “embedding curve;”
- Fix a BOS location;
- Arbitrarily assign curve points to rows;
- Is the attention pattern possibly a solution?

Assumptions:

“Limit” solutions ($d = 2$)

- Fix an “embedding curve;”
- Fix a BOS location;
- Arbitrarily assign curve points to rows;
- Is the attention pattern possibly a solution?

Assumptions:

- ① Each row has a single embedding point;

“Limit” solutions ($d = 2$)

- Fix an “embedding curve;”
- Fix a BOS location;
- Arbitrarily assign curve points to rows;
- Is the attention pattern possibly a solution?

Assumptions:

- 0 Each row has a single embedding point;
- 1 No row can “overpower” the self-attention of another row;

“Limit” solutions ($d = 2$)

- Fix an “embedding curve;”
- Fix a BOS location;
- Arbitrarily assign curve points to rows;
- Is the attention pattern possibly a solution?

Assumptions:

- 0 Each row has a single embedding point;
- 1 No row can “overpower” the self-attention of another row;
- 2 The ratio between self-attention and BOS-attention should be uniform;

Characterizing Limit Solutions

Theorem

The only limit solutions in $d = 2$ have attention patterns given by:

Characterizing Limit Solutions

Theorem

The only limit solutions in $d = 2$ have attention patterns given by:

- Embedded rows lie on a 90° hyperbola;

Characterizing Limit Solutions

Theorem

The only limit solutions in $d = 2$ have attention patterns given by:

- Embedded rows lie on a 90° hyperbola;
- Attention flips (and scales) the embeddings along the hyperbola's axis;

Characterizing Limit Solutions

Theorem

The only limit solutions in $d = 2$ have attention patterns given by:

- Embedded rows lie on a 90° hyperbola;
- Attention flips (and scales) the embeddings along the hyperbola's axis;

Corollary

Every finite $d = 2$ solution that “generalizes well” approximates the attention pattern described above!

Experimental Verification

Experimental Goals

We use the trained models to find

- whether the learned embeddings, attention, and BOS match the theoretical characterization;

Experimental Goals

We use the trained models to find

- whether the learned embeddings, attention, and BOS match the theoretical characterization;
- when training succeeds as we vary d , d_k , and board size (n, k) ;

Experimental Goals

We use the trained models to find

- whether the learned embeddings, attention, and BOS match the theoretical characterization;
- when training succeeds as we vary d , d_k , and board size (n, k) ;
- the smallest (d, d_k) where the geometric structure appears reliably.

Experimental Goals

We use the trained models to find

- whether the learned embeddings, attention, and BOS match the theoretical characterization;
- when training succeeds as we vary d , d_k , and board size (n, k) ;
- the smallest (d, d_k) where the geometric structure appears reliably.

We train models with *stochastic gradient descent* (SGD): at each step, parameters are “nudged” in the direction that decreases the loss on a randomly drawn minibatch.

Experimental Goals

We use the trained models to find

- whether the learned embeddings, attention, and BOS match the theoretical characterization;
- when training succeeds as we vary d , d_k , and board size (n, k) ;
- the smallest (d, d_k) where the geometric structure appears reliably.

We train models with *stochastic gradient descent* (SGD): at each step, parameters are “nudged” in the direction that decreases the loss on a randomly drawn minibatch.

Key Question

Does SGD recover the signature $(-, +)$ operator and 1D embedding curve predicted by theory?

Methodology

We use a single-layer, single-head attention-only decoder with no positional encodings and prepend the BOS token.

Methodology

We use a single-layer, single-head attention-only decoder with no positional encodings and prepend the BOS token.

Training

- New random sample drawn at every step.
- Hyperparameters fixed; train an ensemble of 10^2 – 10^4 models.
- At validation checkpoints, store the decoder weights and analyze geometry.

Methodology

We use a single-layer, single-head attention-only decoder with no positional encodings and prepend the BOS token.

Training

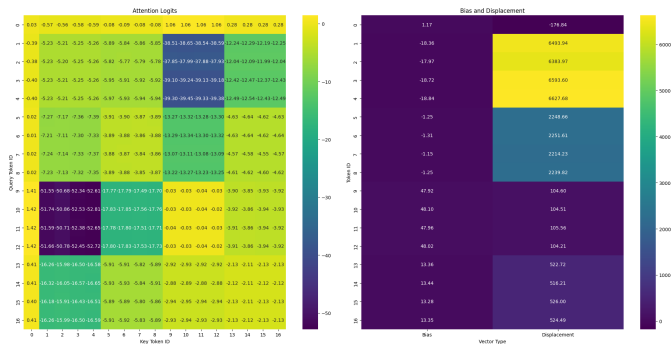
- New random sample drawn at every step.
- Hyperparameters fixed; train an ensemble of 10^2 – 10^4 models.
- At validation checkpoints, store the decoder weights and analyze geometry.

On accuracy

Win-states are sparse (e.g. $\Pr(y=1) \approx \frac{1}{3}$ for $n = 9, k = 3$), so a model predicting “no win” everywhere already achieves $\sim 66\%$ accuracy.

Training with $d_k = 2$

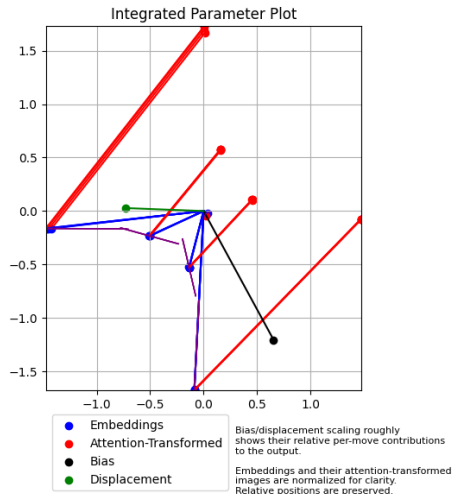
- Attention logits show clear separation between BOS and move tokens
- BOS aligns with the negative direction; winning rows with the positive one.
- Attention induces stable ordering structure



Embedding Geometry at $d_k = 2$

SGD exhibits the expected behavior!

- Token embeddings collapse onto a 1D curve.
- BOS sits as a stable negative offset.
- The operator $W_Q W_K^T$ moves tokens in the predicted directions.
- Same structure appears for all tested (n, k) up to $(16, 4)$.



Solutions at $d_k = 1$

For $n = 9, k = 3$, we occasionally find perfect $d_k = 1$ models (after $\sim 10^7$ steps). However, unlike the $d_k = 2$ case:

- embeddings do not collapse neatly and scatter in 2D

Solutions at $d_k = 1$

For $n = 9, k = 3$, we occasionally find perfect $d_k = 1$ models (after $\sim 10^7$ steps). However, unlike the $d_k = 2$ case:

- embeddings do not collapse neatly and scatter in 2D
- attention does not form a $(-, +)$ operator.

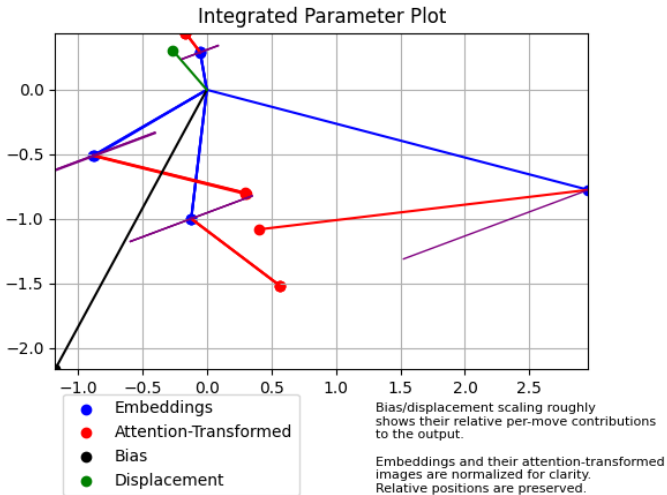
Solutions at $d_k = 1$

For $n = 9, k = 3$, we occasionally find perfect $d_k = 1$ models (after $\sim 10^7$ steps). However, unlike the $d_k = 2$ case:

- embeddings do not collapse neatly and scatter in 2D
- attention does not form a $(-, +)$ operator.
- BOS and row blocks are misaligned with no consistent geometry.

These appear to be sporadic solutions that solve the task by ad-hoc configurations.

Solutions at $d_k = 1$



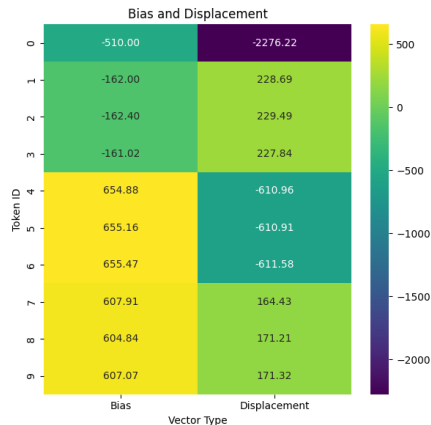
Failure mechanisms

With $d_k = 1$, attention logits have rank 1:

$$s_{ij} = q_i k_j,$$

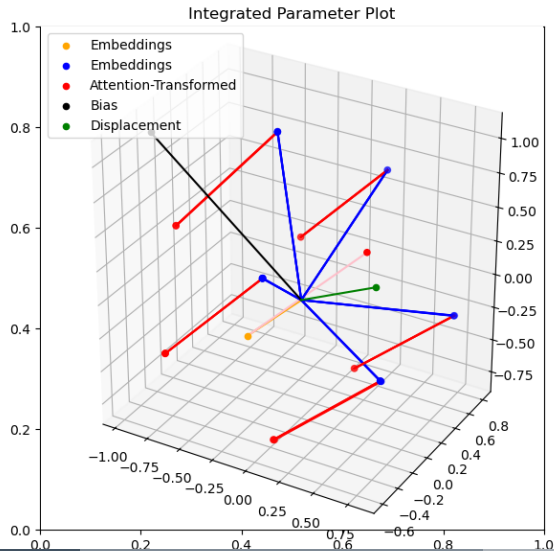
so every token sees the same 1D ordering of keys.

- Model separates BOS and rows along a single axis
- Raising any subset of tokens above BOS necessarily drives others further down
- No 1D ordering can place all row-blocks above BOS simultaneously



Further Results

Dimensionality Reduction - motivation



Experiment

- If we take an existing model, and reduce its key and embedding dimension by 1, through truncated SVD, can we obtain a new perfect model?

Experiment

- If we take an existing model, and reduce its key and embedding dimension by 1, through truncated SVD, can we obtain a new perfect model?
- If not is it well pre-trained?

Truncated SVD

- If $M \in \mathbb{R}^{m \times n}$ then $M = U\Sigma V^T$
 - ▶ $U \in \mathbb{R}^{m \times m}$
 - ▶ $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_{\min(m,n)}) \in \mathbb{R}^{m \times n}$ where $\sigma_i \in \mathbb{R}_{\geq 0}$.
 - ▶ $V \in \mathbb{R}^{n \times n}$

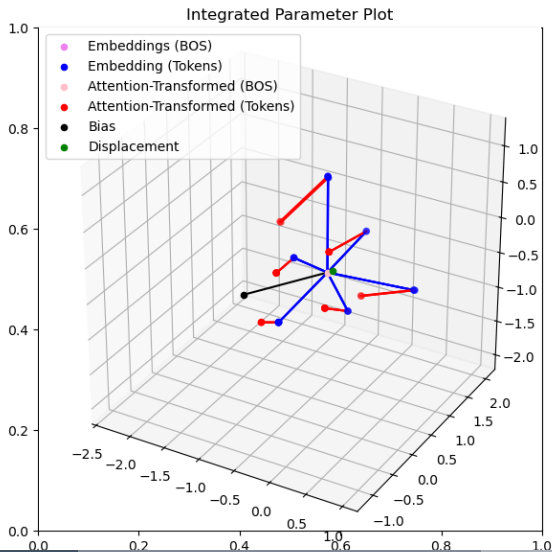
Truncated SVD

- If $M \in \mathbb{R}^{m \times n}$ then $M = U\Sigma V^T$
 - $U \in \mathbb{R}^{m \times m}$
 - $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_{\min(m,n)}) \in \mathbb{R}^{m \times n}$ where $\sigma_i \in \mathbb{R}_{\geq 0}$.
 - $V \in \mathbb{R}^{n \times n}$
- We want M into $M' \in \mathbb{R}^{m \times r}$ with $r < n$ where M' is the subspace of M that captures the most variance.

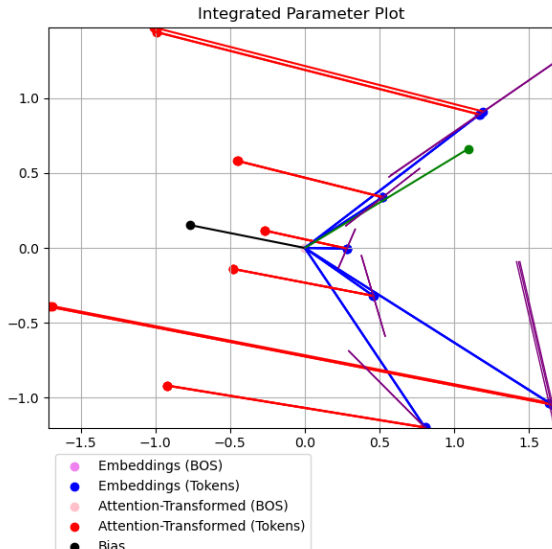
Truncated SVD

- If $M \in \mathbb{R}^{m \times n}$ then $M = U\Sigma V^T$
 - $U \in \mathbb{R}^{m \times m}$
 - $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_{\min(m,n)}) \in \mathbb{R}^{m \times n}$ where $\sigma_i \in \mathbb{R}_{\geq 0}$.
 - $V \in \mathbb{R}^{n \times n}$
- We want M into $M' \in \mathbb{R}^{m \times r}$ with $r < n$ where M' is the subspace of M that captures the most variance.
- Take the first r rows of V , that is $V' \in \mathbb{R}^{r \times n}$ then $M_r = U\Sigma V_r^T = M V_r^T$.

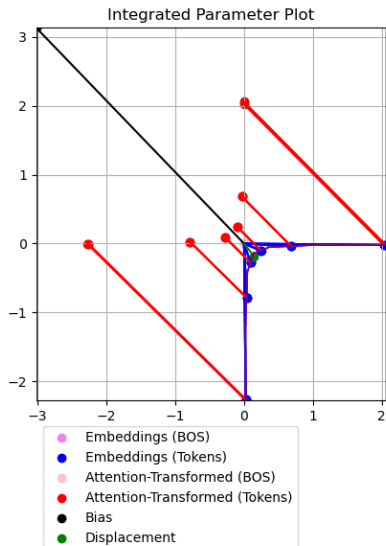
Results - Existing model



Results - SVD on model



Results - Post training



Thank you!

Questions?